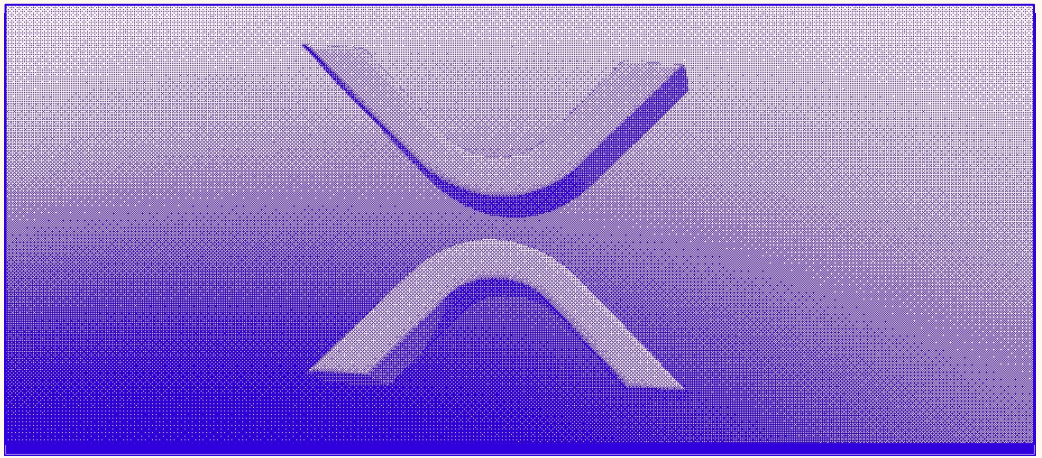
Commons

INTRODUCTION TO XRPL COMMONS

XRPL AI Hack Training • with UKCBT • October 2026





CC BY SA

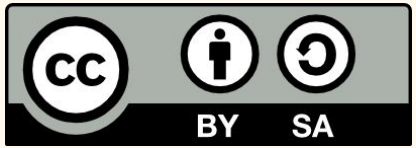
Commons

You are holding open source content.

Here's how to handle it:

→ You can use this content in your work, adapt, and share it.

→ You must mention XRPL Commons and the Creative Commons license as a source.



Creative Commons Attribution-ShareAlike CC BY-SA - This license allows you to protect, reuse and adapt this content even for commercial purposes, if you mention XRPL Commons as a source and allow your re-adapted content to be under the same Open Source License.



Commons

OUR MISSION IS TO CREATE
THE CONDITIONS OF SUCCESS
FOR BUILDERS TO THRIVE
IN THE XRPL ECOSYSTEM.

CC BY SA

Commons

XRPL Commons

Learn

The learning curve for Blockchain is steep. It's better to be supported whether you have tech or business roles.

Build

Our in-house incubator and hackathons offer builders a playground for invention.

Launch

Accelerate your project at the Aquarium Incubator, where developers and entrepreneurs transform ideas into reality.

Adopt

Blockchain integration for corporations and large institutions with tailored support and resources.

Engage

Meetups, podcasts, our Community Magazine and news on our socials unite the XRPL ecosystem.

CC BY SA

Commons

Learn

Teach

Democratize access to knowledge on blockchain technology

chaise
blockchain skills for Europe

Research

Support research development of the XRP Ledger

ripple

Chaire UNESCO ITEN Innovation, Transmission, Edition Numériques

ILB Institut Louis Bachelier

isae SUPAERO Institut Supérieur de l'Aéronautique et de l'Espace

Network

Campus Ambassador Program

KRYPTOSPHERE®

TUM BLOCKCHAIN CLUB

CC BY SA

Commons

BUILD

Our in-house trainings and hackathons offer builders a playground for invention.

Developers

Core Developers

- → Developer Trainings
- → Hackathons
- → XRPL tools & products
- → Infrastructure & Servers
- → Retroactive Funding
- → Core Dev Bootcamp

CC BY SA

Commons

Where this course lives

school

Free journeys

Today's course is a journey on the Academy you can follow at your own pace.

quiz

Quizzes

A short check at the end of each module, so you know you've really got it.

terminal

Real exercises

Python you run yourself against the live XRP Ledger. No wallet needed.

groups

Mentors

Live trainings, office hours, and people who review your work.

CC BY SA

Commons

UKCBT · XRPL AI HACK · MANDATORY TRAINING

Laundering detection on the XRPL

From your first block to a ranked list of accounts worth a closer look.

CC BY SA

Commons

Today's plan

0:00	Welcome, then Part 1: Foundations of Distributed Ledgers	concepts
0:20	Part 2: XRP Ledger Architecture & Transaction Model	concepts
0:50	Part 3: Accessing & Working with Public On-Chain Data	hands-on
1:25	Part 4: Money Laundering Patterns on Public Ledgers	domain
1:50	Part 5: Practical Guidance & Resources, then Q&A	the day

CC BY SA

Commons

The challenge · 24 October

Build an AI/ML model that detects money-laundering-style activity on the XRP Ledger, using real, public on-chain data.

Your data, your tools

You pull the data yourself, pick your own stack, and design your approach from scratch.

Teams of up to 5

Or fly solo. Bring your laptop. IDEALondon, 69 Wilson Street, from 09:30.

£10,000 prize pool

£5,000 for 1st, £3,500 for 2nd, and £1,500 for 3rd place.

CC BY SA

Commons

PART 1 · FOUNDATIONS · 0:00 – 0:20

Distributed Ledgers & the XRPL

What a blockchain is, and why the XRP Ledger is built the way it is.

Academy module: Introduction to the XRP Ledger

CC BY-SA · XRPL Commons

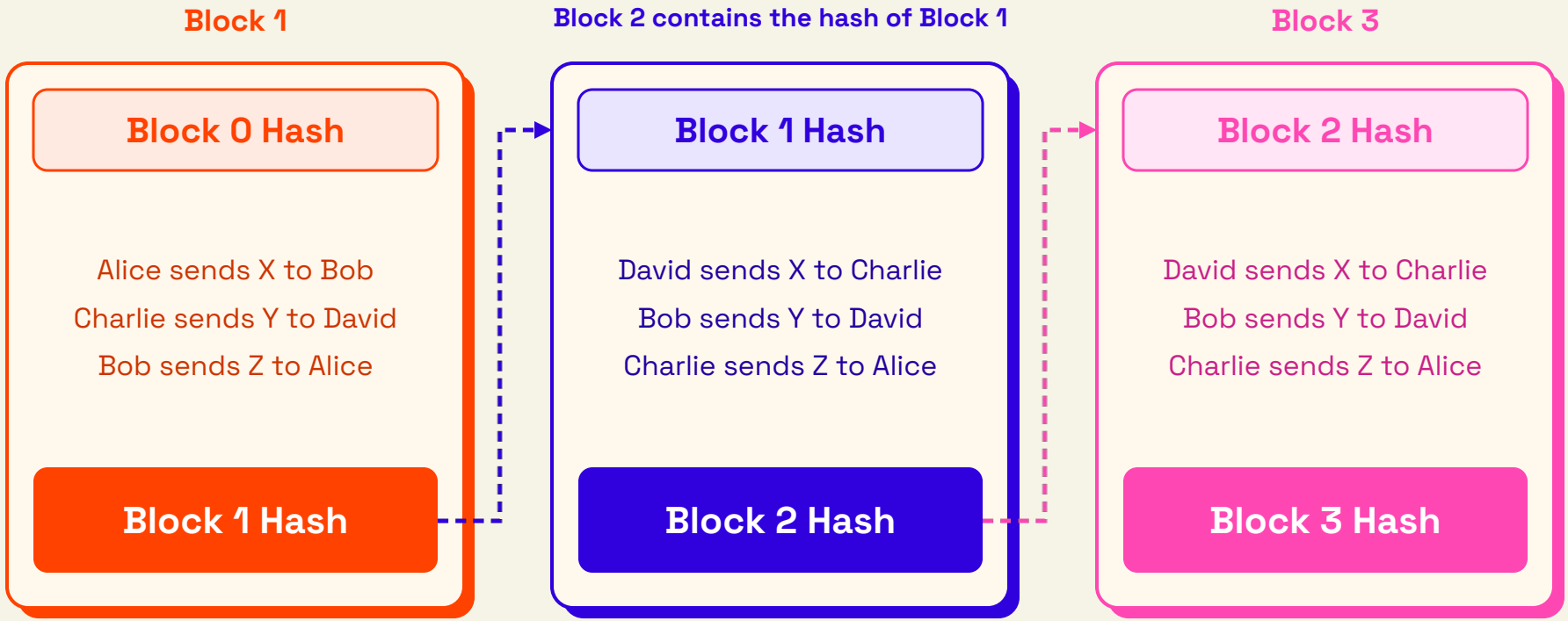
Commons

WHAT IS A BLOCKCHAIN ?

CC BY SA

Commons

Blockchain is a chain of information



CC BY SA

Commons

An Immutable Data Store

The ledger is immutable.

Write Once

Information is recorded permanently on-chain. Once committed, entries can never be modified or erased.

Read Forever

The complete historical record remains accessible, transparent, and queryable by anyone forever.

Tamper-Proof

Data is tamper-proof. Cryptographic hashing ensures absolute integrity across all network nodes.

CC BY SA

Commons

The XRP Ledger

A battle-tested, high-performance Layer-1 blockchain built for global payments.

Open Source

Transparent and publicly accessible code.

Decentralized

Global network powered by independent validator nodes.

Safe & Reliable

Operating continuously for 12+ years without interruption.

Coding Libraries

Robust SDKs for JS, Python, Java, and C++ developers.

Low Carbon

Energy-efficient consensus protocol with minimal footprint.

Fast Settlement

Transactions achieve instant finality in just 3 to 5 seconds.

CC BY SA

Commons

Built for payments since 2012

The XRPL isn't a general-purpose "world computer", it was designed around sending money, issuing currencies and trading assets.

The XRP Ledger

Layer-1 blockchain, open source, decentralized, running non-stop since 2012.

XRP

XRP is the native asset, similar to ETH for Ethereum or SOL for Solana. The primary utility of XRP is to facilitate transactions on the network.

Ripple

A company that builds payment products on top of the XRPL. Ripple is one of many companies contributing to the ledger.

CC BY SA

Commons

Accounts, reserves & tokens

To do anything you need a funded account. The ledger carries two kinds of value: native XRP and issued tokens.



Reserves

Every account locks a small XRP reserve: **1 XRP base + 0.2 XRP per object** owned. It stays yours, just locked.



XRP = cash

Native to the ledger, held by anyone, owes nothing to anyone. Universally accepted.



Tokens = trust

Issued currencies you opt into with a trust line from a chosen counterparty, up to a limit you set.

CC BY SA

Commons

The built-in DEX & consensus

Trading and agreement are part of the protocol, no separate exchange contracts, no mining.

Decentralized exchange

The world's first on-chain DEX.

- **Shared Order Book:** Auto-bridging through XRP & pathfinding across currencies.
- **Native AMM:** Liquidity pools built right into protocol level.
- **MEV Protection:** Eliminates front-running by design.

Consensus without mining

Eco-friendly & high-speed agreement.

- **Trusted UNL Validators:** Agree on state every 3–5 seconds.
- **80%+ Threshold:** Fast finality requiring supermajority agreement.
- **Ultra-Low Cost:** Near-zero energy & fees a fraction of a cent.

CC BY SA

Commons

PART 2 · 0:20 – 0:50

XRP Ledger Architecture & Transaction Model

How a transaction is built, checked and stored, and how to read it as data.

Academy modules: Transactions on the XRP Ledger · Reading XRPL Transactions as Data

CC BY-SA · XRPL Commons

Commons

Every transaction, the same envelope

1. **Account**
   The address that signed it. Its signature authorises the change.
2. **TransactionType**
   Payment, OfferCreate, AMMDeposit… it decides which other fields exist.
3. **Fee**
   A tiny amount of XRP, destroyed, not paid to anyone. It deters spam.
4. **Sequence**
   A per-account counter. It orders transactions and stops replays.
5. **Type-specific fields**
   A Payment adds Destination and Amount. Same envelope, different payload.

CC BY SA

Commons

Three checks before the ledger changes

1. PREFLIGHT

Local checks

Fields present, types right, values sensible.

Cheap rejection

Bad input stops

2. PRECLAIM

Read-only checks

Does the account exist? Can it pay the fee?

Context

Reads state, never writes

3. DOAPPLY

The only writer

Balances move, objects change.

State change

Written to meta

CC BY SA

Commons

What's inside a ledger version

Header

Sequence number, its own hash, its parent's hash, and the close time.

State map

Everything that exists right now: accounts, trust lines, offers, pools.

Transactions

Every transaction applied in this ledger, each with its metadata.

Rules & fees

The fee and reserve settings in force when it closed.

CC BY SA

Commons

Two kinds of data

STATE

Right now

The current snapshot of the ledger

- **Read with:** account_info, account_lines, account_objects.
- **Use it for:** enrichment: balances, flags, holdings.
- **Think of it as:** a balance sheet.

HISTORY

How it got here

Every transaction, ledger after ledger

- **Read with:** ledger and account_tx.
- **Use it for:** patterns, flows and timing.
- **Think of it as:** a bank statement.

Laundering patterns live in history · state enriches them

You build your graph from transactions, then use account settings and labels to make it smarter.

CC BY SA

Commons

The ledger objects that matter

AccountRoot	An account: its XRP balance, sequence, and flags like RequireDest.	nodes
RippleState	A trust line: where issued tokens, like stablecoins, really sit.	tokens
Offer · AMM	DEX orders and pools: value changes asset without a Payment.	conversion
Escrow · Check	Value locked now and released later, outside a short time window.	delayed

CC BY SA

Commons

Which transactions move value

Payment

Direct or cross-currency. The workhorse of every flow.

OfferCreate

A DEX trade. Balances change with no Payment between them.

AMM swaps

Value goes into a pool and can come out as another asset.

Escrow · Check

Value that leaves now and arrives later.

NFT sales

NFTokenAcceptOffer moves value at any price you choose.

AccountDelete

Sweeps an account's remaining XRP somewhere else.

CC BY SA

Commons

Metadata · what actually happened

Trust the **meta**, not the fields the sender signed.

payment.json

```
{
  "tx_json": {
    "TransactionType": "Payment",
    "Account": "rSender...",
    "Destination": "rExchange...",
    "DestinationTag": 4061234,
    "DeliverMax": "250000000"
  },
  "meta": {
    "TransactionResult": "tesSUCCESS",
    "delivered_amount": "250000000",
    "AffectedNodes": [ ... ]
  },
  "close_time_iso": "2026-10-16T08:10:00Z"
}
```

What to read

1. **TransactionResult** tesSUCCESS means it applied. tec means it failed but still paid a fee.
2. **delivered_amount** what really arrived. Never trust Amount or DeliverMax.
3. **DestinationTag** which customer, at a shared exchange address.
4. **AffectedNodes** every balance before and after, DEX and AMM trades included.
5. **close_time_iso** a ready-made timestamp. Raw times count from 1 Jan 2000.

CC BY SA

Commons

Read every amount correctly

XRP

A string of drops. "250000000" is 250 XRP, since 1 XRP = 1,000,000 drops.

Issued token

{currency, issuer, value}. USD from two issuers is two different assets.

MPT

A Multi-Purpose Token: an mpt_issuance_id plus a value.

Ripple time

Seconds since 1 Jan 2000. Add 946684800 to get Unix time.

CC BY SA

Commons

Five traps that corrupt an analysis

1. **Partial payments**
   Amount says 500, delivered_amount says 5. Always read what arrived.

2. **Failed transactions stay on the ledger**
   A tec result paid a fee but moved nothing. Filter it, but keep it as a behaviour.

3. **One address is not one person**
   Exchanges serve thousands of customers at one address, split by destination tag.

4. **Dust and spam**
   Tiny payments sent to thousands of accounts. They drown a raw graph.

5. **Value moves without a Payment**
   DEX trades, AMM swaps, escrows and NFT sales all change balances.

CC BY SA

Commons

PART 3 · 0:50 – 1:25 · HANDS-ON

Accessing & Working with Public On-Chain Data

Pull real Mainnet data yourself. No wallet, no keys, no XRP needed.

CC BY-SA · XRPL Commons

Commons

Where the data lives

s2.ripple.com	Ripple, full history. JSON-RPC at https://s2.ripple.com:51234/	default
xrplcluster.com	InFTF, full history. Enforces a per-minute usage quota.	full history
s1.ripple.com	Ripple's general cluster. Recent history only.	recent
XRPScan · Bithomp	Explorer APIs with account labels. Free tiers have daily limits.	labels

CC BY SA

Commons

Four methods you'll use

ledger

Ledger Data

Everything in ledger N.

Add transactions and expand to get full details.

account_tx

Account History

Everything that touched one account.

Page by page with a marker.

account_info

Account State

An account's balance, flags, and sequence.

Current real-time state.

account_lines

Trustlines & Tokens

Which tokens an account holds.

Includes details on issuing accounts.

CC BY SA

Commons

Download without getting blocked

1. **Pin your range**
   Fixed ledger numbers: cheaper for the server, reproducible for you.
2. **Cache everything**
   Write CSV or Parquet. Train from disk, never from live calls.
3. **Back off on tooBusy**
   Wait, retry, slow down. Every exercise script already does this.
4. **Spread the load**
   Teammates can use different servers. The venue Wi-Fi is shared.
5. **Start small**
   Twenty ledgers first. Check the output, then scale up.

CC BY SA

Commons

Exercise 1 · snapshot one ledger

Fetch the latest validated ledger with **every transaction expanded**.

ex1_ledger_snapshot.py

```python
from collections import Counter
from xrpl.clients import JsonRpcClient
from xrpl.models.requests import Ledger, ServerInfo

client = JsonRpcClient("https://s2.ripple.com:51234/")
info = client.request(ServerInfo()).result["info"]
latest = info["validated_ledger"]["seq"]

ledger = client.request(Ledger(ledger_index=latest,
    transactions=True, expand=True)).result["ledger"]

types, results = Counter(), Counter()
for entry in ledger["transactions"]:
    types[entry["tx_json"]["TransactionType"]] += 1
    results[entry["meta"]["TransactionResult"]] += 1
print(latest, types.most_common(3), results)
```

What the code does

1. **JsonRpcClient** connects to a public full-history Mainnet server.
2. **ServerInfo** finds the latest validated ledger.
3. **expand=True** returns full transactions, not just their hashes.
4. **tx_json + meta** what was signed, and what actually happened.
5. **Counter** tallies types and results. Expect offers, payments and a few tec.

CC BY SA

Commons

Paging through history with a marker

account_tx gives you one page at a time. The marker says there's more. This is Exercise 2.

Ask

Call account_tx with a fixed ledger range and limit=200.

Follow

Got a marker? Send the same query again, with that marker.

Stop

No marker means you have it all. Markers expire, so don't save them.

CC BY SA

Commons

Exercise 3 · build a value-flow graph

Sweep 50 ledgers and turn **successful XRP payments** into a graph.

ex3_flow_graph.py

```
edges = []
for index in range(latest - 50 + 1, latest + 1):
    ledger = request(Ledger(ledger_index=index,
        transactions=True, expand=True))["ledger"]
    for e in ledger["transactions"]:
        tx, meta = e["tx_json"], e["meta"]
        if tx["TransactionType"] != "Payment": continue
        if meta["TransactionResult"] != "tesSUCCESS": continue
        amount = meta.get("delivered_amount")
        if isinstance(amount, str):          # XRP, in drops
            edges.append((tx["Account"], tx["Destination"],
                          int(amount) / 1_000_000))

G = nx.DiGraph()
for src, dst, xrp in edges:
    w = G.get_edge_data(src, dst, {"xrp": 0})["xrp"]
    G.add_edge(src, dst, xrp=w + xrp)
```

What the code does

1. **Ledger sweep** one call per ledger, over a fixed range.
2. **tesSUCCESS only** failed payments moved nothing.
3. **delivered_amount** a string means XRP in drops. Tokens are objects.
4. **DiGraph** one weighted edge per sender and receiver pair.
5. **Look for** dust spammers, big receivers, and accounts that forward what they get.

CC BY SA

LET'S PULL REAL DATA

CC BY SA

Commons

PART 4 · DOMAIN KNOWLEDGE · 1:25 – 1:50

Laundering Patterns on Public Ledgers

What investigators actually look for, and how to turn it into features.

Academy module: Domain Knowledge: Money Laundering Patterns on Public Ledgers

CC BY-SA · XRPL Commons

Commons

WHAT IS MONEY LAUNDERING?

CC BY SA

Commons

Placement, layering, integration

1. PLACEMENT

Getting in

Illicit funds first land in an account.

On-chain signal

A fresh inflow

2. LAYERING

Hiding the trail

Many hops, accounts and assets to blur the origin.

On-chain signal

Fast hops, splits, swaps

3. INTEGRATION

Cashing out

Value comes back as 'clean' money, often via an exchange.

On-chain signal

Tagged deposits

CC BY SA

Commons

FATF red flags that become features

1. **Structuring**
   Many small transfers that look designed to stay under thresholds.
2. **Rapid movement**
   Funds hop through several accounts with no economic reason.
3. **Many accounts, one owner**
   Several addresses that appear to be controlled by the same person.
4. **Risky counterparties**
   Flows to or from high-risk services or obfuscation tools.
5. **Out of character**
   Activity that doesn't match the account's own history.

CC BY SA

Commons

Six shapes analysts look for

Peel chain

A long path of fresh accounts, each peeling a little off.

Fan-out · fan-in

One sum split across many accounts, then gathered again.

Pass-through

Receives and forwards almost everything within minutes.

Round-tripping

Value leaves and comes back through intermediaries.

Asset hopping

XRP to a token and back, or across chains, to lose the tracer.

Fresh-account batch

New accounts funded by one parent, used once, deleted.

CC BY SA

Commons

How they show up on the XRPL

Pass-through	Payment in, near-equal Payment out, a few ledgers later.	seconds
Fresh batches	Same funding parent, minutes apart, later an AccountDelete.	parent
Asset hopping	Cross-currency Payments, OfferCreate and AMM swaps in between.	DEX · AMM
Cash-out	Payments to known exchange addresses, each with a destination tag.	exit

CC BY SA

Commons

What public incidents teach us

Jan 2024 · 213M XRP

Stolen from Chris Larsen's personal accounts. Flagged on-chain within hours, then sent on to several exchanges.

Jun 2019 · GateHub

About 23 million XRP reported stolen from users of the wallet service.

Apr 2025 · xrpl npm

Malicious versions of the official xrpl package sent private keys to an attacker.

CC BY SA

Commons

Three layers, most explainable first

There's no labelled XRPL laundering dataset. So start with what you can explain.

Features

Per account: degree, totals, pass-through ratio, time to forward.

Rules

Readable flags: forwarded 90% within 10 minutes, 10+ uniform splits.

Model

Isolation Forest to rank the unusual. Graph methods if you get labels.

CC BY SA

Commons

Exercise 4 · score accounts by typology

Typologies become **features, rules and a ranking**.

ex4_typology_features.py

```
value = df[df.xrp >= 0.001]                    # set dust aside
out = value.groupby("src").agg(out_xrp=("xrp", "sum"),
                               fan_out=("dst", "nunique"))
inc = value.groupby("dst").agg(in_xrp=("xrp", "sum"),
                               fan_in=("src", "nunique"))
f = out.join(inc, how="outer").fillna(0)

lo = f[["in_xrp", "out_xrp"]].min(axis=1)
hi = f[["in_xrp", "out_xrp"]].max(axis=1)
f["pass_through"] = (lo / hi).fillna(0)      # mule-like hop
f["flag_fan_out"] = f.fan_out >= 10          # smurfing out
f["flag_pass"] = f.pass_through >= 0.9

X = np.log1p(f[["out_xrp", "in_xrp", "fan_out", "fan_in"]])
model = IsolationForest(random_state=42).fit(X)
f["anomaly"] = -model.score_samples(X)       # higher = odder
print(f.sort_values("anomaly", ascending=False).head(10))
```

What the code does

1. **Dust aside** tiny spam payments would drown the signal.
2. **fan_in · fan_out** distinct counterparties: consolidation and smurfing.
3. **pass_through** how much was forwarded: the mule signal.
4. **flag_…** rules anyone can read in one line.
5. **IsolationForest** ranks by unusualness. A review queue, not a verdict.

CC BY SA

Commons

Pitfalls to avoid

1. **Exchanges look anomalous**
   Hot wallets have huge fan-in and fan-out. Give known services their own class.
2. **Dust dominates**
   Remove, or model separately, the payments that carry no real value.
3. **Windows cut patterns in half**
   A peel chain spread over two days is invisible in a one-hour window.
4. **Leakage**
   If your labels come from a rule, don't report that rule's accuracy as your model's.
5. **Naming and shaming**
   Never call an address criminal. Say 'flagged for review' and show the evidence.

CC BY SA

Commons

PART 5 · 1:50 – 2:00

Practical Guidance & Resources

How to scope your project, and how to pitch it.

Academy module: Practical Guidance & Resources for the XRPL AI Hack

CC BY-SA · XRPL Commons

Commons

Proving it works

1. **Case studies**
   Walk through two or three flagged accounts, with real transaction hashes.
2. **Known-incident replay**
   Would your approach have flagged a publicly reported flow?
3. **Synthetic injection**
   Plant a fake peel chain or fan-out, and measure whether you find it.
4. **Stability**
   Does your ranking hold up across different time windows?
5. **False positives**
   Show what you deliberately don't flag, and why.

CC BY SA

Commons

What to present

1 · Problem & data

Which behaviour you target, and what you pulled: from where, how much, how you cleaned it.

2 · Approach & evidence

Features, rules and model in plain words, then one or two real flagged cases.

3 · Limits & next steps

What you'd do with labels, more time or more data. Honesty scores.

CC BY SA

Commons

Analyse addresses

Do

Say "flagged for review". Show the evidence. Say where every label came from.

Never

Call an address criminal, try to identify a person, or paste a seed in a notebook.

Why

Behind every address is a real person. And reading the ledger needs no keys at all.

CC BY SA

Commons

QUESTIONS?

CC BY SA

Commons

THANK YOU

See you tomorrow

Thanks to UKCBT, UCL CBT, Imperial DSS, KCL Blockchain Society, GirlsWhoML, Ripple and the XRPL Commons team.

Tag us: @XRPL_Commons on all platforms

CC BY SA